

Lab Exercise 1: Triggering Nested Loop Join

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.first_name, c.last_name, r.rental_date
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE c.customer_id = 1;
```

dvdrental/postgres@PostgreSQL 16

No limit

Query
Query History

```

1  --EXPLAIN (ANALYZE, BUFFERS )
2  EXPLAIN (ANALYZE, BUFFERS)
3  SELECT c.first_name, c.last_name, r.rental_date
4  FROM customer c
5  JOIN rental r ON c.customer_id = r.customer_id
6  WHERE c.customer_id = 1;

```

Data Output
Messages
Notifications

SQL

Showing rows: 1 to 13
Page No: 1 of 1

	QUERY PLAN
	text
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.771..5.026 rows=32 loops=1)
2	Buffers: shared hit=1 read=152
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.377..0.377 rows=1 loops=1)
4	Index Cond: (customer_id = 1)
5	Buffers: shared hit=1 read=2
6	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.388..4.632 rows=32 loops=1)
7	Filter: (customer_id = 1)
8	Rows Removed by Filter: 16012
9	Buffers: shared read=150
10	Planning:
11	Buffers: shared hit=72 read=5 dirtied=2
12	Planning Time: 5.926 ms
13	Execution Time: 5.788 ms

2. Answer & Explain Analysis Questions

- Determine Roles (Outer vs. Inner):**
 - Outer Table (Outer Loop):** `customer` table.
 - Inner Table (Inner Loop):** `rental` table.

- **Why do this?** PostgreSQL applies Golden Rule 1: The table with the smallest number of result rows after filtering (`WHERE c.customer_id = 1`) becomes the Outer table. Iterating only once in the outer loop is significantly more cost-effective than iterating over thousands of rows.
- **Why choose Nested Loop instead of Hash Join?**
 - The condition `WHERE c.customer_id = 1` returns exactly 1 row from the `customer` table.
 - When the row count of the Outer table is extremely small (only 1 row), creating a Hash Table consumes unnecessary RAM resources and setup time. Optimizing with Nested Loop simply takes that single row and performs a direct lookup.
- **Performance & `loops = 1` metric at the Inner Table node:**
 - The `loops = 1` metric at the Inner node (`rental`) indicates that PostgreSQL only needs to execute the search in the `rental` table exactly once (because the Outer loop runs only once).
 - Combined with the existing Index on the `customer_id` column in the `rental` table, each lookup becomes an extremely fast Index scan.

Lab 2: Thực hành Hash Join

- dvdrental/postgres@PostgreSQL 16

No limit

Query Query History

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT f.title, l.name as language
3 FROM film f
4 JOIN language l ON f.language_id = l.language_id;
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 14 Page No: 1 of 1

	QUERY PLAN text	
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.770..4.056 rows=1000 loops=1)	
2	Hash Cond: (f.language_id = l.language_id)	
3	Buffers: shared hit=1 read=88	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.328..3.220 rows=1000 loo...)	
5	Buffers: shared hit=1 read=87	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.419..0.420 rows=6 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared read=1	
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.173..0.175 rows=6 loo...)	
10	Buffers: shared read=1	
11	Planning:	
12	Buffers: shared hit=161 read=2 dirtied=1	
13	Planning Time: 3.027 ms	
14	Execution Time: 4.179 ms	

- **Probe Phase (Lookup):** The `film` table is scanned row by row to check against the newly created Hash Table (Outer Table).
- **Why do this?** The `language` table contains significantly less data than the `film` table, allowing the Hash Table to consume as little RAM (`work_mem`) as possible.
- **What happens if `work_mem` is reduced to 64kB (`SET work_mem = '64kB';`)?**
 - **Symptom:** The Hash Table no longer fits entirely in RAM.
 - **Why/Consequence:** PostgreSQL has to split the Hash Table and spill temporary data to disk (Spill to Disk, showing `Batches > 1`). This severely degrades performance due to Disk I/O bottlenecks.

Lab 3: Thực hành Merge Join

- **Why is that?** Because the `film` table already has a primary key/index on `film_id`, so PostgreSQL can just scan the index (`Index Scan`) directly to retrieve pre-sorted data without running a `Sort` algorithm.
-
- **Why does removing `ORDER BY` revert the plan back to `Hash Join`?**
 - **Merge Join** only performs efficiently when data on both sides is sorted.
 -
 - Under normal conditions without `ORDER BY`, PostgreSQL prefers building a hash table in RAM (**Hash Join**) because it is faster for unsorted data (like a "zipper" join approach).
 -

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

1. Force a Sequential Scan by disabling Index Scans

dvdrental/postgres@PostgreSQL 16

No limit

Query
Query History

```

1  SET enable_indexscan = off;
2
3  EXPLAIN ANALYZE
4  SELECT * FROM rental WHERE rental_date > '2005-05-24';

```

Data Output
Messages
Notifications

SQL

Showing rows: 1 to 4
Page No: 1 of 1

	QUERY PLAN
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.015..3.310 rows=16044 loop...
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Planning Time: 1.224 ms
4	Execution Time: 4.215 ms

2. Re-enable and compare the Cost and Time

dvdrental/postgres@PostgreSQL 16

No limit

Query Query History

```

1 SET enable_indexscan = on;
2
3 EXPLAIN ANALYZE
4 SELECT * FROM rental WHERE rental_date > '2005-05-24';
    
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 4 Page No: 1 of 1

#	QUERY PLAN text	
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.011..2.892 rows=16044 loop...	
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)	
3	Planning Time: 0.120 ms	
4	Execution Time: 3.799 ms	

Task 2: Join Strategy Analysis

Query

Query History

1

EXPLAIN (ANALYZE, BUFFERS)

2

SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id

3

FROM customer c

4

JOIN rental r ON c.customer_id = r.customer_id

5

JOIN inventory i ON r.inventory_id = i.inventory_id;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 21

✎

🖨️

Page No: 1 of 1

⏪ ⏩ ⏴ ⏵

	QUERY PLAN
	text
1	Merge Join (cost=2422.88..2702.38 rows=16044 width=25) (actual time=18.686..22.850 rows=16044 loops=1)
2	Merge Cond: (c.customer_id = r.customer_id)
3	Buffers: shared hit=16049 read=44
4	-> Index Scan using customer_pkey on customer c (cost=0.28..37.63 rows=599 width=17) (actual time=0.037..0.040 rows=599 loops=1)
5	Buffers: shared hit=6 read=7
6	-> Sort (cost=2422.60..2462.71 rows=16044 width=14) (actual time=18.673..19.698 rows=16044 loops=1)
7	Sort Key: r.customer_id
8	Sort Method: quicksort Memory: 1011kB
9	Buffers: shared hit=16043 read=37
10	-> Merge Join (cost=0.57..1301.94 rows=16044 width=14) (actual time=0.037..14.126 rows=16044 loops=1)
11	Merge Cond: (r.inventory_id = i.inventory_id)
12	Buffers: shared hit=16043 read=37
13	-> Index Scan using idx_fk_inventory_id on rental r (cost=0.29..960.94 rows=16044 width=14) (actual time=0.037..14.126 rows=16044 loops=1)
14	Buffers: shared hit=16039 read=26
15	-> Index Only Scan using inventory_pkey on inventory i (cost=0.28..129.00 rows=4581 width=4) (actual time=0.037..14.126 rows=4581 loops=1)
16	Heap Fetches: 0
17	Buffers: shared hit=4 read=11
18	Planning:
19	Buffers: shared hit=18 read=9

Query Query History



```
1 ANALYZE customer;
2 ANALYZE rental;
3 ANALYZE inventory;
4
5 EXPLAIN (ANALYZE, BUFFERS)
6 SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
7 FROM customer c
8 JOIN rental r ON c.customer_id = r.customer_id
9 JOIN inventory i ON r.inventory_id = i.inventory_id;
```

Data Output Messages Notifications



Showing rows: 1 to 21 Page No: 1 of 1

	QUERY PLAN text
1	Merge Join (cost=2422.88..2702.38 rows=16044 width=25) (actual time=3.847..5.062 rows=16044 loops=1)
2	Merge Cond: (c.customer_id = r.customer_id)
3	Buffers: shared hit=16093
4	-> Index Scan using customer_pkey on customer c (cost=0.28..37.63 rows=599 width=17) (actual time=0.000..0.001 rows=599 loops=1)
5	Buffers: shared hit=13
6	-> Sort (cost=2422.60..2462.71 rows=16044 width=14) (actual time=3.844..4.195 rows=16044 loops=1)
7	Sort Key: r.customer_id
8	Sort Method: quicksort Memory: 1011kB
9	Buffers: shared hit=16080
10	-> Merge Join (cost=0.57..1301.94 rows=16044 width=14) (actual time=0.009..2.779 rows=16044 loops=1)
11	Merge Cond: (r.inventory_id = i.inventory_id)
12	Buffers: shared hit=16080
13	-> Index Scan using idx_fk_inventory_id on rental r (cost=0.29..960.94 rows=16044 width=14) (actual time=0.000..0.001 rows=16044 loops=1)
14	Buffers: shared hit=16065
15	-> Index Only Scan using inventory_pkey on inventory i (cost=0.28..129.00 rows=4581 width=4) (actual time=0.000..0.001 rows=4581 loops=1)